CS 4501 Strategy Design 3: Sponsored Search Auctions

Matheus Venturyne Xavier Ferreira
University of Virginia
Out Wednesday March 18th, 2026
Due **5pm** sharp: **Friday April 3rd, 2026**
Submissions to Canvas

This is a group assignment to be completed by groups of **up to 2 students each**. While you are permitted to discuss strategies and experiments with other students as much as you like, each group must write its own code and explanations. Please include a short statement at the top of your writeup detailing anyone who contributed ideas or debugging help. Both partners must contribute to all aspects of the assignment. You must write all of the code yourselves. If you use any code whatsoever from any source other than standard Python libraries, you **must** acknowledge the source in your writeup. Please also follow the course GenAI policy as stated in the syllabus. Tournament details and any associated announcements will be shared later.

Goal

In this assignment you will design and test bidding agents for a simulated sponsored-search auction. You will implement a balanced-bidding agent, complete the payment rule for a VCG auction, analyze revenue behavior under different auction formats and reserve prices, and then build a budget-aware competition agent for the class tournament.

Setup

Tools: Make sure Python 3 works on your system. The simulator code will not run in Python 2.

Obtaining materials: Download the programming-code release for Strategy Design 3 from Canvas and unpack it. You should have files such as `auction.py`, `gsp.py`, `vcg.py`, `truthful.py`, `bbagent_template.py`, `history.py`, `util.py`, and `start.py`.

Starter files: Pick a unique team name, ideally based on your initials or some short group label. Then run `python start.py TEAMNAME` (or `python3 start.py TEAMNAME` if needed). Replace `TEAMNAME` with your actual team name. For example, if your team name is `abxy`:

```
> python start.py abxy
Copying bbagent_template.py to abxybb.py...
Copying bbagent_template.py to abxybudget.py...
All done.  Code away!
```

This creates the two main agent files you will edit. You will also need to complete the auction-payment code in `vcg.py`.

How the simulator works: The simulator runs repeated sponsored-search auctions over many rounds. In each round, agents submit bids, the mechanism allocates slots, click counts are realized for those slots, and payments are computed. The history of previous rounds is available to your agent, including prior bids, slot assignments, clicks, and payments. Bids and budgets are represented in integer cents.

Testing: Use `python auction.py -h` to see the command-line options. A useful simple test is

```
> python auction.py --iters 20 Truthful,5
```

A good starting point for the experiments in this handout is to vary `--perms`, `--seed`, `--iters`, `--reserve`, `--budget`, and `--mech` as directed below. For mixed populations, you can include multiple agent types on the command line, for example

```
> python auction.py --perms 10 --iters 200 Truthful,4 abxybb,1
```

Problem Set

1. **The Balanced-Bidding Agent**

   (a) In `TEAMNAMEbb.py`, write an agent that best-responds to the previous-period bids of the other agents using a *balanced bidding* strategy. You are given a truthful bidding agent in `truthful.py` and a starter template in `bbagent_template.py`. Consider period t. Let b_{-i}^{t-1} denote the bids from the agents other than i in period $t-1$. Suppose there are m positions. Balanced bidding proceeds as follows:

   - Given bids b_{-i}^{t-1}, agent i targets the position j^* that maximizes

     $$\max_{j \in \{1,\ldots,m\}} \left[pos_j \cdot (v_i - price_j) \right],$$

     where pos_j is the position effect and $price_j$ is the price the agent would pay for position j given bids b_{-i}^{t-1}. In this simulator, you should estimate pos_j using the number of clicks from the previous round. In GSP with no reserve price, $price_j$ equals the jth-highest bid in b_{-i}^{t-1}. Your utility calculation should omit your own previous bid and should account for the reserve price.
   - *Not expecting to win:* If $price_{j^*} \geq v_i$ in the target position, then bid $b_i^t = v_i$ in period t.

- Otherwise:
  (a) *Not going for the top:* If $j^* > 1$, then choose b_i^t so that

$$pos_{j^*}(v_i - price_{j^*}) = pos_{j^*-1}(v_i - b_i^t).$$

  (b) *Going for the top:* If $j^* = 1$, bid $b_i^t = v_i$.

This strategy is motivated by the balanced-bidding discussion in Section 10.4.2 of the reading. You will need to complete the code that computes expected utility for each slot and then uses that to compute the bid. In the provided code, bids from the previous round can be accessed through `history.round(t-1)`; the helper `slot_info` already extracts useful information for you.

(b) **Experimental analysis.** For the following questions, run the simulation with 5 agents. Leave the budget at its default value of \$5000; it should not bind in this part.

i. What is the average utility of a population of truthful agents? What is the average utility of a population of balanced-bidding agents? Compare the two cases and explain your findings.
A good starting point is to use `--perms 1 --seed 2 --iters 200`.

ii. What is the average utility of one balanced-bidding agent against 4 truthful agents, and of one truthful agent against 4 balanced-bidding agents? For this experiment you should run multiple permutations. For example:

```
> python auction.py --perms 10 --iters 200 Truthful,4 abxybb,1
```

What do your results suggest about the incentives to follow the truthful strategy versus the balanced-bidding strategy?

2. **Experiments with Revenue: GSP vs. VCG Auctions**

In this section you will compare the revenue properties of the GSP and VCG auctions under different reserve prices. A *reserve price*, $r > 0$, sets a minimum price for any position. Used carefully, reserve prices can increase revenue. In GSP, only bids weakly above r can be allocated. The lowest allocated bidder pays the maximum of the reserve price and the highest unallocated bid. The VCG auction works similarly, and one way to think about the reserve is as if there were a dummy bidder at bid r, while bids below r are ignored.

(a) Complete the code in `vcg.py` that implements the VCG payment rule. The allocation rule is already implemented. Because of the reserve price, it is most convenient to use the recursive form of the VCG payment rule (see the proof of Theorem 10.3 in the reading).

In particular, suppose there are 3 bidders with bids weakly above the reserve price, and $b_1 \geq b_2 \geq b_3$, with b_4 the fourth-highest bid overall. Bidders 1–3 get the top 3 positions. Let $t_{\text{vcg},i}(b)$ denote the expected payment by bidder i in a single auction given bid profile b. For bidder 3,

$$t_{\text{vcg},3}(b) = pos_3 \max(r, b_4).$$

For bidders 1 and 2, this can be calculated recursively as

$$t_{\text{vcg},i}(b) = (pos_i - pos_{i+1})b_{i+1} + t_{\text{vcg},i+1}(b).$$

(b) Run all of the following simulations with 5 agents and leave the budget at its default value of \$5000.

i. What is the auctioneer's revenue under GSP with no reserve price when all agents use the balanced-bidding strategy? What happens as the reserve price increases? What is the revenue-optimal reserve price?
Use `--reserve INT` to set the reserve price in cents. A good starting point is `--perms 1 --iters 200`.

ii. What is the auctioneer's revenue under VCG with no reserve price when all agents are truthful? What happens as the reserve price increases? Explain your findings and compare them with your GSP results.
Again, `--perms 1 --seed 2 --iters 200` is a good starting point.

iii. Fix the reserve price to zero. Explore what might happen if a search engine switched from GSP to VCG in the middle of the process. To do this, run balanced-bidding agents in GSP and then switch to VCG at period 24 using the `--mech=switch` option. What happens to revenue?
Again, `--perms`, `--seed`, and `--iters` are all useful here.

iv. In one paragraph, state what you learned from these exercises about agent design, auction design, and revenue. There is no single right answer here; focus on what your experiments showed you.

3. **The Competition: Budget-Aware Agent**

(a) In `TEAMNAMEbudget.py`, write your competition agent. This agent will compete in the simulated GSP auction against the agents submitted by other groups. The balanced-bidding agent from earlier ignores the budget constraint; your goal here is to design an agent that is explicitly budget-aware.

For full credit, your agent must run in the auction simulation and be bug-free. **All code needed to run the competition agent must be inside `TEAMNAMEbudget.py`; do not rely on helper files.**

In your writeup, describe in a few sentences how your agent works, why you designed it that way, and how you expect it to perform in the class competition. You are *not* expected to spend many hours finding a perfect strategy. Consider a few ideas, test them, and submit the best version you have.

A few possible directions you could explore are:

- Bid less aggressively when click-through rates are low.
- Spend more when competition is weak and prices are lower.
- Try to exploit the budget limits of other agents by forcing them to spend earlier.

Important: if your agent throws an error during a round of the tournament, it will be treated as if it bid zero in that round.

(b) Tournament details and any associated announcements will be shared later.

For the tournament itself, the daily budget constraint will be set to $600. We will run a GSP auction with a small reserve price, usually in groups of about 5 agents at a time. The planned tournament structure is Swiss-style over 8 rounds: in the first round agents are placed randomly into groups of 5, and in later rounds they are grouped with agents that have the same prior track record.

The core tournament settings will be based on something close to

```
python auction.py --perms 24 --num-rounds 48 --mech=gsp \
  --iters 100 --budget 60000
```

followed by the list of submitted agents. Minor adjustments may be made if needed for runtime or stability.

Submission Instructions

Only one group member needs to make the submission. That group member should upload the following files (replacing `TEAMNAME` with your actual team name) to the Canvas assignment. Make sure all group members' names are listed on the writeup:

1. `TEAMNAMEbb.py` — your balanced-bidding agent.
2. `TEAMNAMEbudget.py` — your budget-aware competition agent.
3. `vcg.py` — your completed VCG payment-rule implementation.
4. `TEAMNAMEsolution.pdf` — your written answers to the analysis and experimental questions, plus a short description of your competition agent.

Please submit the files themselves directly. **Do NOT submit a single `.zip` file containing the folder.**

Comments and Hints

Read the baseline code: Before writing much code, skim the main simulator and baseline files, especially `auction.py`, `truthful.py`, `bbagent_template.py`, `gsp.py`, `vcg.py`, and `history.py`.

Use small tests first: Debug on short runs with a small number of agents before moving to the larger experiments.

Watch units: Values, bids, reserve prices, payments, and budgets are all stored in integer cents.

Run enough iterations: Many of the statistics in this assignment are noisy. Use enough permutations and iterations that your comparisons are stable.

Do not modify the simulator: Your grade is based on code that works with the unmodified simulator. If you changed any core files while experimenting, re-download a clean copy and re-test before submitting.

No hidden dependencies: Especially for the competition part, do not rely on extra helper files beyond the required submission files.

Questions: If you have any questions, reach out to Jou.